

Team Note of BunCua Team

MinhhHoangPersia - Pain - Lingg

Compiled on September 14, 2026

Contents

1	Template	1
2	Data Structure	1
2.1	Segment Tree Walk	1
2.2	Persistent Segment Tree	1
2.3	Trie	2
2.4	Persistent Trie	2
2.5	Fenwick Tree	3
2.6	Fenwick Tree 2D	3
2.7	XOR Basis	3
3	Graphs	5
3.1	2 SAT	5
3.2	LCA O(1)	5
3.3	Centroid Decomposition	5
3.4	Heavy Light Decomposition	6
3.5	Bellman Ford	6
3.6	Floyd	6
3.7	Ford Fulkerson	7
3.8	Dinic	8
3.9	Min Cost Max Flow	9
3.10	Dsu On Tree(Sack)	10
4	DP	11
4.1	DP Digit Optimization 1	11
4.2	DP Digit Optimization 2	11
4.3	DP SoS	12
5	Strings	12
6	Math	12
6.1	Extended Euclid	12
7	Geometry	12
8	Miscellaneous	12

1 Template

2 Data Structure

2.1 Segment Tree Walk

```

/* Find first element in range [u, v] have value >= val */
int findPos(int id, int l, int r, int u, int v, ll val) {
    if(l > v || r < u) return n + 1;
    if(st[id] < val) return n + 1;
    if(l == r) return l;
    push(id);
    int mid = (l + r) / 2;
    int res = findPos(id * 2, l, mid, u, v, val);
    if(res == n + 1) {
        res = findPos(id * 2 + 1, mid + 1, r, u, v, val);
    }
    return res;
}

```

2.2 Persistent Segment Tree

```

const int N = 3e5 + 5;
const int LOG = 20;
const int MAX_NODES = N * 40;

struct Node {
    int left, right, v;
    Node(int _left = 0, int _right = 0, int _v = 0) {
        left = _left;

```

```

        right = _right;
        v = _v;
    }
} st[MAX_NODES];

int cnt = 0;
int ver[N], nver = 0;

void refine(int id) {
    st[id].v = st[st[id].left].v + st[st[id].right].v;
}

int build(int l, int r) {
    if (l == r) {
        cnt++;
        st[cnt] = Node(0, 0, 0);
        return cnt;
    }
    int mid = (l + r) >> 1;
    int cur = ++cnt;
    st[cur].left = build(l, mid);
    st[cur].right = build(mid + 1, r);
    refine(cur);
    return cur;
}

int update(int id, int l, int r, int u, int x) {
    if (l == r) {
        cnt++;
        st[cnt] = Node(0, 0, st[id].v + x);
        return cnt;
    }
    int mid = (l + r) >> 1;
    int cur = ++cnt;
    if (u <= mid) {
        st[cur].left = update(st[id].left, l, mid, u, x);
        st[cur].right = st[id].right;
    } else {
        st[cur].left = st[id].left;
        st[cur].right = update(st[id].right, mid + 1, r, u, x);
    }
    refine(cur);
    return cur;
}

int get(int id, int l, int r, int u, int v) {
    if (l > v || r < u) return 0;
    if (l >= u && r <= v) return st[id].v;
    int mid = (l + r) >> 1;
    return get(st[id].left, l, mid, u, v) + get(st[id].right,
        mid + 1, r, u, v);
}

void add_version(int l, int r, int u, int x) {
    nver++;
    ver[nver] = update(ver[nver - 1], l, r, u, x);
}

/*
    USAGE EXAMPLE (Solving Distinct Values in Range - DQUERY):

    // 1. Initialize base tree
    cnt = nver = 0;
    ver[0] = build(1, n);

    // 2. Build versions tracking the latest occurrence of each
    value
    for (int i = 1; i <= n; i++) {
        // If value appeared before, remove its previous position
        if (mark[a[i]] > 0) {
            ver[nver] = update(ver[nver], 1, n, mark[a[i]], -1);
        }
    }

```

```

// Add current position
nver++;
ver[nver] = update(ver[nver - 1], 1, n, i, 1);

iver[i] = ver[nver]; // iver[i] stores the version for
prefix i
mark[a[i]] = i;
}

// 3. Query distinct values in [L, R]
int ans = get(iver[R], 1, n, L, R);
*/

```

2.3 Trie

```

const int LOG_MAX = 29;
const int MAX_NODES = 3e5 * 32;

struct Node {
    int c[2];
    int cnt;

    Node() {
        c[0] = c[1] = 0;
        cnt = 0;
    }
};

Node trie[MAX_NODES];
int node_cnt = 0;

void init_trie() {
    for (int i = 0; i <= node_cnt; i++) {
        trie[i] = Node();
    }
    node_cnt = 0;
}

void insert(int val) {
    int cur = 0;
    for (int i = LOG_MAX; i >= 0; i--) {
        int x = (val >> i) & 1;
        if (trie[cur].c[x] == 0) {
            trie[cur].c[x] = ++node_cnt;
        }
        cur = trie[cur].c[x];
        trie[cur].cnt++;
    }
}

bool search(int val) {
    int cur = 0;
    for (int i = LOG_MAX; i >= 0; i--) {
        int x = (val >> i) & 1;
        cur = trie[cur].c[x];
        if (cur == 0 || trie[cur].cnt == 0) {
            return false;
        }
    }
    return trie[cur].cnt > 0;
}

int count_occurrences(int val) {
    int cur = 0;
    for (int i = LOG_MAX; i >= 0; i--) {
        int x = (val >> i) & 1;
        cur = trie[cur].c[x];
        if (cur == 0 || trie[cur].cnt == 0) {
            return 0;
        }
    }
    return trie[cur].cnt;
}

void erase(int val) {
    if (!search(val)) return;
    int cur = 0;
    for (int i = LOG_MAX; i >= 0; i--) {
        int x = (val >> i) & 1;
        cur = trie[cur].c[x];

```

```

        trie[cur].cnt--;
    }
}

void update(int old_val, int new_val) {
    if (!search(old_val)) return;
    erase(old_val);
    insert(new_val);
}

```

2.4 Persistent Trie

```

const int LOG_MAX = 29;
const int N = 1e5 + 5;
const int MAX_NODES = N * 32;

int t[MAX_NODES][2];
int num[MAX_NODES];
int root[N];
int cnt = 0;

void init_trie() {
    for (int i = 0; i <= cnt; i++) {
        t[i][0] = t[i][1] = num[i] = 0;
    }
    cnt = 0;
    root[0] = 0;
}

int newnode() {
    cnt++;
    t[cnt][0] = t[cnt][1] = num[cnt] = 0;
    return cnt;
}

int add_version(int prev_root, int val) {
    int cur_root = newnode();
    int cur = cur_root;
    int old = prev_root;
    for (int i = LOG_MAX; i >= 0; i--) {
        int x = (val >> i) & 1;
        t[cur][0] = t[old][0];
        t[cur][1] = t[old][1];
        t[cur][x] = newnode();
        cur = t[cur][x];
        old = t[old][x];
        t[cur][0] = t[old][0];
        t[cur][1] = t[old][1];
        num[cur] = num[old] + 1;
    }
    return cur_root;
}

// 4. Query Maximum XOR with 'val' using combinations of
// versions.
// Example: Query on a tree path from u to v with LCA p,
// parent of p is pp.
// The effective count of a node is: num[x] + num[y] - num[p]
// - num[pp].
int query_max_xor(int rootX, int rootY, int rootP, int rootPP,
int val) {
    int res = 0;
    int x = rootX, y = rootY, p = rootP, pp = rootPP;
    for (int i = LOG_MAX; i >= 0; i--) {
        int v = (val >> i) & 1;
        int count_opposite = num[t[x][v ^ 1]] + num[t[y][v ^ 1]]
            - num[t[p][v ^ 1]] - num[t[pp][v ^ 1]];
        if (count_opposite > 0) {
            res |= (1 << i);
            x = t[x][v ^ 1];
            y = t[y][v ^ 1];
            p = t[p][v ^ 1];
            pp = t[pp][v ^ 1];
        } else {
            x = t[x][v];
            y = t[y][v];
            p = t[p][v];
            pp = t[pp][v];
        }
    }
    return res;
}

```

```

}

/*
USAGE EXAMPLE (Max XOR in an array from L to R):

// 1. Build prefix versions
init_trie();
for (int i = 1; i <= n; i++) {
    root[i] = add_version(root[i - 1], a[i]);
}

// 2. Query Max XOR with 'val' in subarray [L, R]
// For arrays, the effective tree is root[R] - root[L-1].
// You can use a modified query function that only takes 2
roots:
// count_opposite = num[t[rootR][v ^ 1]] -
num[t[rootL_minus_1][v ^ 1]];
*/

```

2.5 Fenwick Tree

```

struct Fenwick {
    int n;
    vector<long long> bit;

    Fenwick(int _n) {
        n = _n;
        bit.assign(n + 1, 0);
    }

    void update(int idx, long long delta) {
        for (; idx <= n; idx += idx & -idx) {
            bit[idx] += delta;
        }
    }

    long long query(int idx) {
        long long sum = 0;
        for (; idx > 0; idx -= idx & -idx) {
            sum += bit[idx];
        }
        return sum;
    }

    long long query(int l, int r) {
        if (l > r) return 0;
        return query(r) - query(l - 1);
    }

    // O(log N) Bit Jumping: Finds the FIRST index where prefix
    sum >= val.
    // Note: This only works if all elements in the BIT are
    non-negative!
    int lower_bound(long long val) {
        if (val <= 0) return 1;
        int pos = 0;
        long long sum = 0;
        for (int i = __lg(n); i >= 0; i--) {
            int next_pos = pos + (1 << i);
            if (next_pos <= n && sum + bit[next_pos] < val) {
                sum += bit[next_pos];
                pos = next_pos;
            }
        }
        // pos is the largest index such that prefix sum < val
        // So pos + 1 is the first index such that prefix sum >=
        val
        return pos + 1;
    }
};

```

2.6 Fenwick Tree 2D

```

/*
Idea:
- 1-based indexing for a grid of size N x M.
- When we add `v` to a 2D difference array at (i, j), it
  contributes to
  the prefix sum S(X, Y) (where X >= i and Y >= j) by:
  v * (X - i + 1) * (Y - j + 1)

```

```

= v * [ (X+1)(Y+1) - (X+1)*j - (Y+1)*i + i*j ]
- To maintain this, we need 4 2D BITS to store the sum of:
1. v
2. v * i
3. v * j
4. v * i * j
*/

struct Fenwick2D {
    int n, m;
    vector<vector<long long>> bit1, bit2, bit3, bit4;

    Fenwick2D(int _n, int _m) {
        n = _n;
        m = _m;
        bit1.assign(n + 2, vector<long long>(m + 2, 0));
        bit2.assign(n + 2, vector<long long>(m + 2, 0));
        bit3.assign(n + 2, vector<long long>(m + 2, 0));
        bit4.assign(n + 2, vector<long long>(m + 2, 0));
    }

    void add(int x, int y, long long val) {
        for (int i = x; i <= n; i += i & -i) {
            for (int j = y; j <= m; j += j & -j) {
                bit1[i][j] += val;
                bit2[i][j] += val * x;
                bit3[i][j] += val * y;
                bit4[i][j] += val * x * y;
            }
        }
    }

    void update(int x1, int y1, int x2, int y2, long long val) {
        add(x1, y1, val);
        add(x2 + 1, y1, -val);
        add(x1, y2 + 1, -val);
        add(x2 + 1, y2 + 1, val);
    }

    // Internal Prefix Query: gets the sum of submatrix from (1,
    1) to (x, y)
    long long query_prefix(int x, int y) {
        long long res = 0;
        for (int i = x; i > 0; i -= i & -i) {
            for (int j = y; j > 0; j -= j & -j) {
                res += bit1[i][j] * (x + 1) * (y + 1)
                    - bit2[i][j] * (y + 1)
                    - bit3[i][j] * (x + 1)
                    + bit4[i][j];
            }
        }
        return res;
    }

    // Range Query: gets the sum of submatrix from (x1, y1) to
    (x2, y2)
    long long query(int x1, int y1, int x2, int y2) {
        return query_prefix(x2, y2)
            - query_prefix(x1 - 1, y2)
            - query_prefix(x2, y1 - 1)
            + query_prefix(x1 - 1, y1 - 1);
    }
};

```

2.7 XOR Basis

```

/*
XOR BASIS (Linear Basis over GF(2))

Idea:
- Represents a set of numbers using a minimal set of
  linearly independent
  numbers (basis).
- Any XOR sum of a subset in the original set can be formed
  by a XOR sum
  of a subset of the basis.
- The size of the basis never exceeds the number of bits
  (LOG_D).

Complexity:
- Insert / Check / Get Max: O(LOG_D)

```

```

- Merge two bases:  $O(\text{LOG\_D}^2)$ 
- Build RREF:  $O(\text{LOG\_D}^2)$ 
- K-th Smallest/Largest:  $O(\text{LOG\_D})$ 
*/

#include <bits/stdc++.h>
using namespace std;

const int LOG_D = 60; // 60 for values up to  $10^{18}$ , 30 for values up to  $10^9$ 

struct XorBasis {
    long long basis[LOG_D];
    int sz; // Number of elements currently in the basis
    vector<long long> reduced_basis; // Stores the canonical form for K-th queries

    XorBasis() {
        memset(basis, 0, sizeof(basis));
        sz = 0;
    }

    // 1. Insert a value. Returns true if successfully inserted
    // (i.e., the value is linearly independent from the current basis).
    bool insert(long long x) {
        for (int i = LOG_D - 1; i >= 0; i--) {
            if ((x >> i) & 1) {
                if (!basis[i]) {
                    basis[i] = x;
                    sz++;
                    return true;
                }
                x ^= basis[i];
            }
        }
        return false; // x became 0, meaning it was already spanned by the basis
    }

    // 2. Check if a value can be formed by XORing a subset of the basis
    bool check(long long x) {
        for (int i = LOG_D - 1; i >= 0; i--) {
            if ((x >> i) & 1) {
                if (!basis[i]) return false;
                x ^= basis[i];
            }
        }
        return true;
    }

    // 3. Get the maximum possible XOR sum using the basis.
    long long get_max(long long start_val = 0) {
        long long res = start_val;
        for (int i = LOG_D - 1; i >= 0; i--) {
            if ((res ^ basis[i]) > res) {
                res ^= basis[i];
            }
        }
        return res;
    }

    // 4. Merge this basis with another basis
    void merge(const XorBasis& other) {
        for (int i = 0; i < LOG_D; i++) {
            if (other.basis[i]) {
                insert(other.basis[i]);
            }
        }
    }

    // K-TH LARGEST / SMALLEST QUERIES
    // Note: These functions only consider DISTINCT values generated by the basis.
    // There are exactly  $2^{\text{sz}}$  distinct values (including 0 from the empty subset).

    // 5. Build Reduced Row Echelon Form (Canonical Form)
    // MUST be called after all inserts and BEFORE querying k-th values!

```

```

void build() {
    reduced_basis.clear();
    for (int i = LOG_D - 1; i >= 0; i--) {
        for (int j = i - 1; j >= 0; j--) {
            // If the j-th bit of basis[i] is set, use basis[j] to turn it off
            if ((basis[i] >> j) & 1) {
                basis[i] ^= basis[j];
            }
        }
    }
    // Collect non-zero elements into the reduced_basis array (in ascending order)
    for (int i = 0; i < LOG_D; i++) {
        if (basis[i]) {
            reduced_basis.push_back(basis[i]);
        }
    }
}

// 6. Get the k-th smallest distinct XOR sum (1-indexed).
// k = 1 returns 0 (the empty subset).
// Returns -1 if k is out of bounds ( $k > 2^{\text{sz}}$ ).
long long get_kth_smallest(long long k) {
    long long total_combinations = 1LL << sz;
    if (k < 1 || k > total_combinations) return -1;

    long long rank = k - 1; // 0-indexed rank
    long long res = 0;
    for (int i = 0; i < sz; i++) {
        if ((rank >> i) & 1) {
            res ^= reduced_basis[i];
        }
    }
    return res;
}

// 7. Get the k-th largest distinct XOR sum (1-indexed).
// k = 1 returns the absolute maximum possible XOR sum.
// Returns -1 if k is out of bounds ( $k > 2^{\text{sz}}$ ).
long long get_kth_largest(long long k) {
    long long total_combinations = 1LL << sz;
    if (k < 1 || k > total_combinations) return -1;

    // The k-th largest is equivalent to the (total_combinations - k)-th smallest
    long long rank = total_combinations - k;
    long long res = 0;
    for (int i = 0; i < sz; i++) {
        if ((rank >> i) & 1) {
            res ^= reduced_basis[i];
        }
    }
    return res;
}
};

/*
USAGE EXAMPLE:

signed main() {
    int n = 3;
    long long a[] = {2, 4, 6}; // 2 (010), 4 (100), 6 (110)

    XorBasis xb;
    for (int i = 0; i < n; i++) xb.insert(a[i]);

    // 1. Build the canonical form before calling k-th queries
    xb.build();

    // The distinct XOR sums generated are: 0, 2, 4, 6
    // Since 6 is just  $2^4$ , the basis size (sz) will be 2. ( $2^2 = 4$  combinations)

    cout << "Max XOR sum: " << xb.get_max() << "\n"; // 6

    cout << "1st smallest: " << xb.get_kth_smallest(1) << "\n"; // 0
    cout << "2nd smallest: " << xb.get_kth_smallest(2) << "\n"; // 2
}
*/

```

```

    cout << "1st largest: " << xb.get_kth_largest(1) << "\n";
    // 6
    cout << "3rd largest: " << xb.get_kth_largest(3) << "\n";
    // 2

    return 0;
}
*/

```

3 Graphs

3.1 2 SAT

```

/*
- For a boolean variable x, we create two nodes: x (1..n)
and NOT(x) (n+1..2n).
- A clause (x OR y) is equivalent to: (!x => y) and (!y =>
x).
- If x and NOT(x) are in the same Strongly Connected
Component (SCC),
it's a contradiction -> NO solution.
- We assign TRUE to x if its SCC is topologically sorted
AFTER NOT(x)'s SCC.
(Tarjan assigns SCC IDs in reverse topological order, so
col[x] < col[NOT(x)])
*/

```

```
const int N = 2e4 + 5;
```

```

int n;
vector<int> G[2 * N];
int num[2 * N], low[2 * N], cnt;
int col[2 * N], ct;
vector<int> stk;
int ans[N];

```

```

int NOT(int x) {
    return (x <= n ? x + n : x - n);
}

```

```

void addedge(int x, int y) {
    G[NOT(x)].push_back(y);
    G[NOT(y)].push_back(x);
}

```

```

void tarjan(int u) {
    num[u] = low[u] = ++cnt;
    stk.push_back(u);
    for (int v : G[u]) {
        if (!col[v]) {
            if (num[v]) {
                low[u] = min(low[u], num[v]);
            } else {
                tarjan(v);
                low[u] = min(low[u], low[v]);
            }
        }
    }
    if (num[u] == low[u]) {
        ct++;
        while (true) {
            int v = stk.back();
            stk.pop_back();
            col[v] = ct;
            if (v == u) break;
        }
    }
}

```

```

bool solve() {
    rep(i, 1, 2 * n) {
        if (!num[i]) tarjan(i);
    }
    rep(i, 1, n) {
        if (col[i] == col[NOT(i)]) return false;
        ans[i] = (col[i] < col[NOT(i)]);
    }
    return true;
}

```

3.2 LCA O(1)

```
const int N = 1e5 + 5;
const int LOG = 18;
```

```

vector<int> G[N];
int tin[N], cnt, rn[N];
int euler[2 * N], ct, first[N];
int depth[N];
int rmq[LOG][2 * N];

```

```

void dfs(int u, int p = 0) {
    tin[u] = ++cnt;
    rn[cnt] = u;
    euler[++ct] = tin[u];
    first[tin[u]] = ct;
    for (int v : G[u]) {
        if (v == p) continue;
        depth[v] = depth[u] + 1;
        dfs(v, u);
        euler[++ct] = tin[u];
    }
}

```

```

void build_lca() {
    for (int i = 1; i <= ct; i++) {
        rmq[0][i] = euler[i];
    }
    for (int j = 1; j < LOG; j++) {
        for (int i = 1; i <= ct - (1 << (j - 1)); i++) {
            rmq[j][i] = min(rmq[j - 1][i], rmq[j - 1][i + (1 << (j - 1))]);
        }
    }
}

```

```

int lca(int x, int y) {
    x = first[tin[x]];
    y = first[tin[y]];
    if (x > y) swap(x, y);
    int k = 31 - __builtin_clz(y - x + 1);
    int min_tin = min(rmq[k][x], rmq[k][y - (1 << k) + 1]);
    return rn[min_tin];
}

```

3.3 Centroid Decomposition

```
const int N = 3e5 + 5;
vector<int> G[N];
int sz[N];
bool deleted[N];
int centroid_par[N];

```

```

void get_sz(int u, int par) {
    sz[u] = 1;
    for (int v : G[u]) {
        if (v != par && !deleted[v]) {
            get_sz(v, u);
            sz[u] += sz[v];
        }
    }
}

```

```

int get_centroid(int u, int par, int total_nodes) {
    for (int v : G[u]) {
        if (v != par && !deleted[v]) {
            if (sz[v] > total_nodes / 2) {
                return get_centroid(v, u, total_nodes);
            }
        }
    }
    return u;
}

```

```

void solve_centroid(int centroid) {
    // TODO: Add your logic here (e.g., DFS to collect all
    // distances from the
    // centroid to the nodes in its component, then count valid
    // pairs).
    //
    // Example structure:

```

```

// for (int v : G[centroid]) {
//     if (!deleted[v]) {
//         // 1. DFS(v) to query answer with currently stored
//         //         paths
//         // 2. DFS(v) again to add new paths to our data
//         //         structure
//     }
// }
// // 3. Clear the data structure for the next centroid
}

void build_centroid(int u, int p = -1) {
    get_sz(u, -1);
    int centroid = get_centroid(u, -1, sz[u]);
    deleted[centroid] = true;
    centroid_par[centroid] = p;
    solve_centroid(centroid);
    for (int v : G[centroid]) {
        if (!deleted[v]) {
            build_centroid(v, centroid);
        }
    }
}

```

3.4 Heavy Light Decomposition

```

const int N = 2e5 + 5;
vector<int> G[N];
int sz[N], son[N], depth[N], pre[N];
int in[N], cnt, top[N];

void dfs1(int u, int par) {
    sz[u] = 1;
    son[u] = 0;
    pre[u] = par;
    for (int v : G[u]) {
        if (v != par) {
            depth[v] = depth[u] + 1;
            dfs1(v, u);
            sz[u] += sz[v];
            if (sz[v] > sz[son[u]]) {
                son[u] = v;
            }
        }
    }
}

void rebuild(int u, int topp) {
    in[u] = ++cnt;
    top[u] = topp;
    if (son[u]) {
        rebuild(son[u], topp);
    }
    for (int v : G[u]) {
        if (v != pre[u] && v != son[u]) {
            rebuild(v, v);
        }
    }
}

int query_path(int x, int y) {
    int res = 0;
    while (top[x] != top[y]) {
        if (depth[top[x]] < depth[top[y]]) {
            swap(x, y);
        }
        res = max(res, get(1, 1, n, in[top[x]], in[x]));
        x = pre[top[x]];
    }
    if (depth[x] > depth[y]) {
        swap(x, y);
    }
    res = max(res, get(1, 1, n, in[x], in[y]));
    return res;
}

void update_path(int x, int y, int val) {
    while (top[x] != top[y]) {
        if (depth[top[x]] < depth[top[y]]) {
            swap(x, y);
        }
    }
}

```

```

x = pre[top[x]];
}
if (depth[x] > depth[y]) {
    swap(x, y);
}
}
/*
USAGE EXAMPLE:
dfs1(1, 1);
cnt = 0;
rebuild(1, 1);
// for(int i = 1; i <= n; i++) update(1, 1, n, in[i],
a[i]);
*/

```

3.5 Bellman Ford

```

const long long LINF = 1e18;

struct Edge {
    int u, v;
    long long w;
};

int n, m;
vector<Edge> edges;
long long dist[2005];
int parent[2005];

bool bellman_ford(int src) {
    for (int i = 1; i <= n; i++) {
        dist[i] = LINF;
        parent[i] = -1;
    }
    dist[src] = 0;
    for (int i = 1; i <= n - 1; i++) {
        bool updated = false;
        for (auto e : edges) {
            if (dist[e.u] < LINF && dist[e.u] + e.w < dist[e.v]) {
                dist[e.v] = dist[e.u] + e.w;
                parent[e.v] = e.u;
                updated = true;
            }
        }
        if (!updated) break;
    }
    bool has_negative_cycle = false;
    for (int i = 1; i <= n; i++) {
        for (auto e : edges) {
            if (dist[e.u] < LINF && dist[e.u] + e.w < dist[e.v]) {
                dist[e.v] = -LINF;
                has_negative_cycle = true;
            }
        }
    }
    return has_negative_cycle;
}

/*
HOW TO EXTRACT A NEGATIVE CYCLE (If needed)
If you only need to print ONE negative cycle (like CSSES
Cycle Finding):
1. Keep an array `parent`.
2. Run Bellman-Ford for exactly N iterations (no need for
Phase 2 above).
3. On the N-th iteration, if an edge (u, v) is relaxed, `v`
is either in
    the negative cycle or reachable from it.
4. To find a node strictly inside the cycle, go back `N`
times using `parent`:
    int x = v;
    for (int i = 1; i <= n; i++) x = parent[x];
5. Now `x` is guaranteed to be in the cycle. Trace back
using `parent[x]`
until you reach `x` again, push nodes to a vector, and
reverse it.
*/

```

3.6 Floyd

```

const long long LINF = 1e18;

```

```

const int N = 505;

long long dist[N][N];
int nxt[N][N];

void init(int n) {
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            dist[i][j] = LINF;
            nxt[i][j] = -1;
        }
        dist[i][i] = 0;
    }
}

void add_edge(int u, int v, long long w) {
    if (w < dist[u][v]) {
        dist[u][v] = w;
        nxt[u][v] = v;
    }
}

void floyd_warshall(int n) {
    for (int k = 1; k <= n; k++) {
        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= n; j++) {
                if (dist[i][k] < LINF && dist[k][j] < LINF) {
                    if (dist[i][k] + dist[k][j] < dist[i][j]) {
                        dist[i][j] = dist[i][k] + dist[k][j];
                        nxt[i][j] = nxt[i][k];
                    }
                }
            }
        }
    }

    // 4. (Optional) Propagate Negative Cycles
    // If node k is part of a negative cycle, any path going
    // through k
    // will also have a length of -Infinity.
    for (int k = 1; k <= n; k++) {
        if (dist[k][k] < 0) {
            for (int i = 1; i <= n; i++) {
                for (int j = 1; j <= n; j++) {
                    if (dist[i][k] < LINF && dist[k][j] < LINF) {
                        dist[i][j] = -LINF;
                    }
                }
            }
        }
    }

    // 5. Path Reconstruction
    // Returns the path from u to v, or an empty vector if no path
    // exists
    vector<int> get_path(int u, int v) {
        if (dist[u][v] == LINF) return {};

        vector<int> path;
        for (int at = u; at != v; at = nxt[at][v]) {
            if (at == -1) return {};
            path.push_back(at);
        }
        path.push_back(v);
        return path;
    }
}

```

3.7 Ford Fulkerson

```

/*
Idea:
- Repeatedly find an augmenting path from Source (S) to
  Sink (T) using BFS.
- Bottleneck of the path is the minimum remaining capacity
  among its edges.
- Push the bottleneck flow through the path, and ADD it to
  the reverse
  edges (to allow "undoing" flow later).
- Terminates when no more augmenting paths can be found.
Complexity:  $O(V * E^2)$ . Safe for  $V \leq 1000$ ,  $E \leq 5000$ .

```

```

/*
const long long LINF = 1e18;

struct Edge {
    int u, v;
    long long cap, flow;
};

struct MaxFlow {
    int n;
    vector<Edge> edges;
    vector<vector<int>>> adj;

    MaxFlow(int _n) {
        n = _n;
        adj.assign(n + 1, vector<int>());
    }

    // Add a directed edge from u to v with capacity `cap`.
    // If it's an undirected edge, set `is_directed = false`.
    void add_edge(int u, int v, long long cap, bool is_directed
= true) {
        adj[u].push_back(edges.size());
        edges.push_back({u, v, cap, 0}); // Forward edge

        adj[v].push_back(edges.size());
        edges.push_back({v, u, is_directed ? 0 : cap, 0}); //
        Backward (residual) edge
    }

    // Find max flow from s to t
    long long max_flow(int s, int t) {
        long long total_flow = 0;
        vector<int> parent_edge(n + 1);

        while (true) {
            // 1. BFS to find an augmenting path
            fill(parent_edge.begin(), parent_edge.end(), -1);
            queue<int> q;
            q.push(s);
            parent_edge[s] = -2; // Mark source as visited

            while (!q.empty() && parent_edge[t] == -1) {
                int u = q.front();
                q.pop();

                for (int id : adj[u]) {
                    int v = edges[id].v;
                    // If v is not visited AND the edge has remaining
                    // capacity
                    if (parent_edge[v] == -1 && edges[id].cap -
                        edges[id].flow > 0) {
                        parent_edge[v] = id; // Store the edge ID used to
                        reach v
                        q.push(v);
                    }
                }
            }

            // 2. If Sink (t) is not reached, no more paths exist ->
            // break
            if (parent_edge[t] == -1) break;

            // 3. Find the bottleneck capacity along the path found
            // by BFS
            long long push = LINF;
            int cur = t;
            while (cur != s) {
                int id = parent_edge[cur];
                push = min(push, edges[id].cap - edges[id].flow);
                cur = edges[id].u;
            }

            // 4. Augment the flow along the path and update
            // residual edges
            cur = t;
            while (cur != s) {
                int id = parent_edge[cur];
                edges[id].flow += push; // Add flow to
                forward edge
                edges[id ^ 1].flow -= push; // Subtract flow
                from backward edge (undo capability)
            }
        }
    }
}

```

```

        cur = edges[id].u;
    }

    total_flow += push;
}
return total_flow;
}

// Get Min Cut: Returns a boolean array where visited[i] =
true means
// vertex `i` belongs to the Source's component (Set S).
vector<bool> get_min_cut(int s) {
    vector<bool> visited(n + 1, false);
    queue<int> q;
    q.push(s);
    visited[s] = true;

    // BFS one last time on edges that are NOT fully saturated
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        for (int id : adj[u]) {
            int v = edges[id].v;
            if (!visited[v] && edges[id].cap - edges[id].flow > 0) {
                visited[v] = true;
                q.push(v);
            }
        }
    }
    return visited;
}

// Tracing: Print all edges that actually carry water in the
final state
void print_flow_paths() {
    cout << "Edges with positive flow:\n";
    // We step by 2 to only check the original forward edges
    (0, 2, 4...)
    for (int i = 0; i < edges.size(); i += 2) {
        if (edges[i].flow > 0) {
            cout << edges[i].u << " -> " << edges[i].v
                << " : " << edges[i].flow << " / " <<
                    edges[i].cap << "\n";
        }
    }
}

};

/*
USAGE EXAMPLE:
signed main() {
    int n = 4; // 4 vertices
    MaxFlow mf(n);

    // add_edge(u, v, capacity)
    mf.add_edge(1, 2, 3);
    mf.add_edge(1, 3, 2);
    mf.add_edge(2, 3, 1);
    mf.add_edge(2, 4, 2);
    mf.add_edge(3, 4, 4);

    int source = 1, sink = 4;
    cout << "Max Flow: " << mf.max_flow(source, sink) <<
        "\n";

    // Print flow network details
    mf.print_flow_paths();

    // Find the Min Cut edges
    vector<bool> in_source_set = mf.get_min_cut(source);
    cout << "Min Cut Edges:\n";
    for (int i = 0; i < mf.edges.size(); i += 2) {
        int u = mf.edges[i].u;
        int v = mf.edges[i].v;
        // An edge is in the Min Cut if it goes from the Source
        Set to the Sink Set
        if (in_source_set[u] && !in_source_set[v]) {
            cout << u << " -> " << v << " (Capacity cut: " <<
                mf.edges[i].cap << ")\n";
        }
    }
}

```

```

    }

    return 0;
}
*/

3.8 Dinic

/*
DINIC'S ALGORITHM (Max Flow & Min Cut)
Idea:
- Phase 1 (BFS): Build a "Level Graph". Assign levels to
nodes based on
    shortest path from Source (S). Only edges that go to the
    next level
    (level[v] == level[u] + 1) are considered valid.
- Phase 2 (DFS): Push as much flow as possible along the
Level Graph.
- DEAD-END OPTIMIZATION (ptr array): During DFS, if an edge
leads to a
    dead end (can't push any more flow), we advance the `ptr`
    so we NEVER
    visit this useless edge again in the current BFS phase.
Complexity:
- General Graph:  $O(V^2 * E)$ 
- Unit Network (e.g., Bipartite Matching):  $O(E * \sqrt{V})$ 
- Extremely fast in practice, much faster than
    Edmonds-Karp.
*/
const long long LINF = 1e18;

struct Edge {
    int u, v;
    long long cap, flow;
};

struct Dinic {
    int n;
    vector<Edge> edges;
    vector<vector<int>>> adj;
    vector<int> level; // Stores the BFS level of each node
    vector<int> ptr; // Optimization: keeps track of the next
    edge to explore in DFS

    Dinic(int _n) {
        n = _n;
        adj.assign(n + 1, vector<int>());
        level.assign(n + 1, -1);
        ptr.assign(n + 1, 0);
    }

    void add_edge(int u, int v, long long cap, bool is_directed
= true) {
        adj[u].push_back(edges.size());
        edges.push_back({u, v, cap, 0});

        adj[v].push_back(edges.size());
        edges.push_back({v, u, is_directed ? 0 : cap, 0});
    }

    // BFS to build the Level Graph
    bool bfs(int s, int t) {
        fill(level.begin(), level.end(), -1);
        level[s] = 0;
        queue<int> q;
        q.push(s);

        while (!q.empty()) {
            int u = q.front();
            q.pop();
            for (int id : adj[u]) {
                int v = edges[id].v;
                // If not visited AND has remaining capacity
                if (level[v] == -1 && edges[id].cap - edges[id].flow >
                    0) {
                    level[v] = level[u] + 1;
                    q.push(v);
                }
            }
        }
    }

    // Return true if the sink is reachable

```

```

    return level[t] != -1;
}

// DFS to push flow along the Level Graph
long long dfs(int u, int t, long long pushed) {
    if (pushed == 0 || u == t) return pushed;

    // Note the reference `&cid = ptr[u]`. This is the core
    // optimization!
    // It prevents checking dead ends multiple times in the
    // same BFS phase.
    for (int& cid = ptr[u]; cid < adj[u].size(); ++cid) {
        int id = adj[u][cid];
        int v = edges[id].v;
        long long tr = edges[id].cap - edges[id].flow; //
        // Remaining capacity

        // Only push flow to the exact next level
        if (level[u] + 1 != level[v] || tr == 0) continue;

        long long push = dfs(v, t, min(pushed, tr));
        if (push == 0) continue;

        edges[id].flow += push;
        edges[id ^ 1].flow -= push;
        return push;
    }
    return 0;
}

// Main function to calculate Max Flow
long long max_flow(int s, int t) {
    long long flow = 0;
    while (bfs(s, t)) { // While a path exists
        fill(ptr.begin(), ptr.end(), 0); // Reset the dead-end
        // pointers

        // Push as much flow as possible in the current Level
        // Graph
        while (long long pushed = dfs(s, t, LINF)) {
            flow += pushed;
        }
    }
    return flow;
}

// Min Cut Extraction: Nodes reachable from Source in the
// residual graph
vector<bool> get_min_cut(int s) {
    vector<bool> visited(n + 1, false);
    queue<int> q;
    q.push(s);
    visited[s] = true;

    while (!q.empty()) {
        int u = q.front();
        q.pop();
        for (int id : adj[u]) {
            int v = edges[id].v;
            if (!visited[v] && edges[id].cap - edges[id].flow > 0)
            {
                visited[v] = true;
                q.push(v);
            }
        }
    }
    return visited; // visited[i] = true means 'i' is in the
    // Source set
}

// Tracing: Print edges that carry flow
void print_flow_paths() {
    cout << "Edges with positive flow:\n";
    for (int i = 0; i < edges.size(); i += 2) { // Check
        // forward edges only
        if (edges[i].flow > 0) {
            cout << edges[i].u << " -> " << edges[i].v
                << " : " << edges[i].flow << " / " <<
                edges[i].cap << "\n";
        }
    }
}

```

```

    }
};

/*
USAGE EXAMPLE:
signed main() {
    int n = 4; // 4 vertices
    Dinic dinic(n);

    // add_edge(u, v, capacity)
    dinic.add_edge(1, 2, 3);
    dinic.add_edge(1, 3, 2);
    dinic.add_edge(2, 3, 1);
    dinic.add_edge(2, 4, 2);
    dinic.add_edge(3, 4, 4);

    int source = 1, sink = 4;
    cout << "Max Flow: " << dinic.max_flow(source, sink) <<
    "\n"; // Output: 5

    // Extract Min Cut
    vector<bool> in_source_set = dinic.get_min_cut(source);
    cout << "Min Cut Edges:\n";
    for (int i = 0; i < dinic.edges.size(); i += 2) {
        int u = dinic.edges[i].u;
        int v = dinic.edges[i].v;
        if (in_source_set[u] && !in_source_set[v]) {
            cout << u << " -> " << v << " (Capacity: " <<
            dinic.edges[i].cap << ")\n";
        }
    }

    return 0;
}
*/

```

3.9 Min Cost Max Flow

```

/*
=====
MIN COST MAX FLOW (Dijkstra with Johnson's Potentials)

Idea:
- Uses Dijkstra instead of SPFA to find the cheapest
  augmenting path.
- Maintains a potential function `pi` for each node to
  handle negative
  weight residual edges.
- Edge weights are reweighted:  $w'(u, v) = w(u, v) + pi[u] - pi[v]$ 
   $\geq 0$ .
- Complexity:  $O(F * E \log V)$  where  $F$  is the flow. Fast and
  safe!
=====
*/
const long long LINF = 1e18;

struct MCMFEdge {
    int u, v;
    long long cap, flow, cost;
};

struct MinCostMaxFlow {
    int n;
    vector<MCMFEdge> edges;
    vector<vector<int>>> adj;
    vector<long long> pi, dist;
    vector<int> parent_edge;

    MinCostMaxFlow(int _n) {
        n = _n;
        adj.assign(n + 1, vector<int>());
        pi.assign(n + 1, 0);
        dist.assign(n + 1, 0);
        parent_edge.assign(n + 1, -1);
    }

    void add_edge(int u, int v, long long cap, long long cost,
        bool is_directed = true) {
        adj[u].push_back(edges.size());
        edges.push_back({u, v, cap, 0, cost});
    }
}

```

```

adj[v].push_back(edges.size());
edges.push_back({v, u, is_directed ? 0 : cap, 0, -cost});
}

// Run SPFA once to set initial potentials.
// ONLY UNCOMMENT inside mcmf() if the original graph has
// NEGATIVE edge costs.
void set_pi(int s) {
    fill(pi.begin(), pi.end(), LINF);
    vector<bool> in_queue(n + 1, false);
    queue<int> q;

    pi[s] = 0;
    q.push(s);
    in_queue[s] = true;

    while (!q.empty()) {
        int u = q.front();
        q.pop();
        in_queue[u] = false;

        for (int id : adj[u]) {
            int v = edges[id].v;
            if (edges[id].cap - edges[id].flow > 0 && pi[v] >
                pi[u] + edges[id].cost) {
                pi[v] = pi[u] + edges[id].cost;
                if (!in_queue[v]) {
                    q.push(v);
                    in_queue[v] = true;
                }
            }
        }
    }
}

// Dijkstra using reduced costs
bool dijkstra(int s, int t) {
    fill(dist.begin(), dist.end(), LINF);
    fill(parent_edge.begin(), parent_edge.end(), -1);

    priority_queue<pair<long long, int>, vector<pair<long
long, int>>, greater<pair<long long, int>>> pq;

    dist[s] = 0;
    pq.push({0, s});

    while (!pq.empty()) {
        long long d = pq.top().first;
        int u = pq.top().second;
        pq.pop();

        if (d > dist[u]) continue;

        for (int id : adj[u]) {
            int v = edges[id].v;
            long long remain_cap = edges[id].cap - edges[id].flow;

            if (remain_cap > 0) {
                long long reduced_cost = edges[id].cost + pi[u] -
                    pi[v];
                if (dist[v] > dist[u] + reduced_cost) {
                    dist[v] = dist[u] + reduced_cost;
                    parent_edge[v] = id;
                    pq.push({dist[v], v});
                }
            }
        }
    }
    return dist[t] != LINF;
}

pair<long long, long long> mcmf(int s, int t, long long
max_required_flow = LINF) {
    long long total_flow = 0;
    long long total_cost = 0;

    // Uncomment this line ONLY IF the initial graph contains
    // negative costs
    // set_pi(s);

    while (total_flow < max_required_flow && dijkstra(s, t)) {

```

```

        long long push = max_required_flow - total_flow;
        int cur = t;
        while (cur != s) {
            int id = parent_edge[cur];
            push = min(push, edges[id].cap - edges[id].flow);
            cur = edges[id].u;
        }

        cur = t;
        while (cur != s) {
            int id = parent_edge[cur];
            edges[id].flow += push;
            edges[id ^ 1].flow -= push;
            total_cost += push * edges[id].cost;
            cur = edges[id].u;
        }

        total_flow += push;

        // Update potentials for the next iteration
        for (int i = 1; i <= n; i++) {
            if (dist[i] != LINF) {
                pi[i] += dist[i];
            }
        }
    }

    return {total_flow, total_cost};
}

//
// -----
// PATH TRACING: Prints all edges that are actively carrying
// flow
// -----
void print_flow_paths() {
    cout << "Edges with positive flow (Min Cost Max Flow):\n";
    // Iterate by 2 to check only the original forward edges
    for (int i = 0; i < edges.size(); i += 2) {
        if (edges[i].flow > 0) {
            cout << edges[i].u << " -> " << edges[i].v
                << " | Flow: " << edges[i].flow << " / " <<
                    edges[i].cap
                << " | Unit Cost: " << edges[i].cost
                << " | Total Edge Cost: " << edges[i].flow *
                    edges[i].cost << "\n";
        }
    }
}
};

```

3.10 Dsu On Tree(Sack)

```

/*
DSU ON TREE (SACK)

Idea:
- Efficiently answers OFFLINE queries about subtrees in  $O(N \log N)$  time.
- Solves problems like: "For each node U, how many nodes in its subtree have a specific property?"

Algorithm:
1. Find the "heavy" child (largest subtree) for each node.
2. DFS traversal:
    - Traverse all "light" children, answer their queries, then CLEAR their data.
    - Traverse the "heavy" child, answer its queries, and KEEP its data.
    - Add the data of the light children and the current node itself.
    - Answer queries for the current node.
    - If the current node is a light child (`keep == false`), CLEAR all its data.
*/

const int N = 2e5 + 5;
vector<int> G[N];

```

```
// Variables for Euler tour and Heavy-Light logic
int in[N], out[N], rn[N], timer;
int sz[N], big[N];

// 1. First DFS: Precompute subtree sizes, Euler tour, and heavy children
void dfs_sz(int u, int par) {
    in[u] = ++timer;
    rn[timer] = u;
    sz[u] = 1;
    big[u] = 0;

    for (int v : G[u]) {
        if (v != par) {
            dfs_sz(v, u);
            sz[u] += sz[v];
            if (sz[big[u]] < sz[v]) {
                big[u] = v; // Update heavy child
            }
        }
    }
    out[u] = timer;
}

// Struct to manage the global state (Modify this according to the problem!)
// Example: Counting frequencies of values
struct GlobalState {
    int freq[N];
    int count_of_freq[N];

    void add(int node) {
        // int val = a[node];
        // count_of_freq[++freq[val]]++;
    }

    void remove(int node) {
        // int val = a[node];
        // count_of_freq[freq[val]--]--;
    }

    // int get_answer(...) { return ...; }
} state;

// 2. Second DFS: DSU on Tree (Sack)
void dsu_on_tree(int u, int par, bool keep) {
    // Step 2.1: Traverse light children and DO NOT keep their data
    for (int v : G[u]) {
        if (v != par && v != big[u]) {
            dsu_on_tree(v, u, false);
        }
    }

    // Step 2.2: Traverse the heavy child and KEEP its data
    if (big[u] != 0) {
        dsu_on_tree(big[u], u, true);
    }

    // Step 2.3: Add light children's data and the current node's data
    for (int v : G[u]) {
        if (v != par && v != big[u]) {
            for (int i = in[v]; i <= out[v]; i++) {
                state.add(rn[i]);
            }
        }
    }
    state.add(u); // Add the current node itself

    // Step 2.4: ANSWER QUERIES for node `u` here
    // for (auto query : queries[u]) {
    //     ans[query.id] = state.get_answer(query.k);
    // }

    // Step 2.5: If we are a light child, clear our data so it doesn't affect
    // other branches of our parent.
    if (!keep) {
        for (int i = in[u]; i <= out[u]; i++) {
            state.remove(rn[i]);
        }
    }
}
```

```
    }
}

/*
USAGE EXAMPLE:
signed main() {
    // ... Read graph and queries ...
    // ... Save queries as `queries[u].push_back({k, query_id})` ...
    timer = 0;
    dfs_sz(1, -1);
    // Start DSU on tree from the root, keeping is irrelevant for the root
    dsu_on_tree(1, -1, false);
    // Print answers
    // ...
    return 0;
}
*/
```

4 DP

4.1 DP Digit Optimization 1

```
/*
DIGIT DP OPTIMIZATION 1: REMOVING THE `is_less` (fr) DIMENSION
*/

vector<int> R;
pair<ll, ll> dp[20];

pair<ll, ll> calc(int pos, int fr) {
    if (pos == -1) return make_pair(0, 1);
    auto &res = dp[pos];
    if (fr && res != make_pair(-1LL, -1LL)) return res;
    int lim_r = (fr ? 9 : R[pos]);
    pair<ll, ll> val = make_pair(0, 0);
    for (int i = 0; i <= lim_r; i++) {
        pair<ll, ll> nxt = calc(pos - 1, fr | (i < lim_r));
        val.first += nxt.first + i * nxt.second;
        val.second += nxt.second;
    }
    return (fr ? res = val : val);
}
```

4.2 DP Digit Optimization 2

```
/*
DIGIT DP OPTIMIZATION 2: ONE-TIME MEMSET FOR MULTI-TEST CASES
*/

const int mod = 1e9 + 7;

string R;
int dp[51][51][51][51];

int calc(int pos, int c3, int c6, int c9, int fr) {
    if (pos == -1) {
        return (c3 > 0 && c3 == c6 && c6 == c9);
    }
    auto &res = dp[pos][c3][c6][c9];
    if (fr && res != -1) return res;
    ll val = 0;
    int lim_r = (fr ? 9 : R[pos] - '0');
    for (int i = 0; i <= lim_r; i++) {
        val += calc(pos - 1, c3 + (i == 3), c6 + (i == 6), c9 + (i == 9), fr | (i < lim_r));
        if (val >= mod) val -= mod;
    }
    return (fr ? res = val : val);
}

int solve(string x) {
    reverse(x.begin(), x.end());
    R = x;
    return calc((int)R.size() - 1, 0, 0, 0, 0);
}
```

```

bool check(string x) {
    int c3 = 0, c6 = 0, c9 = 0;
    for (auto i : x) {
        c3 += (i == '3');
        c6 += (i == '6');
        c9 += (i == '9');
    }
    return (c3 > 0 && c3 == c6 && c6 == c9);
}

/*
// Usage in main():
// memset(dp, -1, sizeof dp); // Call this ONLY ONCE
// while (test--) {
//     string a, b; cin >> a >> b;
//     cout << (1LL * solve(b) - solve(a) + check(a) + mod) %
mod << "\n";
// }
*/

```

4.3 DP SoS

```

const int LOG = 20;
int dp[1 << LOG];

// 1. Sum over Submasks: dp[mask] = \sum_{sub \subseteq mask} A[sub]
// (sub is formed by turning OFF some bits of mask)
void sos_submask(int n, int a[]) {
    for (int mask = 0; mask < (1 << n); mask++) {
        dp[mask] = a[mask];
    }
    for (int i = 0; i < n; i++) {
        for (int mask = 0; mask < (1 << n); mask++) {
            if ((mask >> i) & 1) {
                dp[mask] += dp[mask ^ (1 << i)];
            }
        }
    }
}

// 2. Sum over Supermasks: dp[mask] = \sum_{sup \supseteq mask} A[sup]
// (sup is formed by turning ON some bits of mask)
void sos_supermask(int n, int a[]) {
    for (int mask = 0; mask < (1 << n); mask++) {
        dp[mask] = a[mask];
    }
    for (int i = 0; i < n; i++) {
        for (int mask = 0; mask < (1 << n); mask++) {
            if (!(mask >> i) & 1) {
                dp[mask] += dp[mask ^ (1 << i)];
            }
        }
    }
}

/*
COMMON USAGE PATTERNS & TRICKS:
1. Find pairs (u, v) such that u & v = 0:
2. Number of elements `x` such that (x & mask) == x:
3. Number of elements `x` such that (x & mask) == mask:
4. Finding two elements x, y with maximum (x | y):
*/

```

5 Strings

6 Math

6.1 Extended Euclid

```

// 1. Core Extended GCD
// Returns gcd(a, b) and updates x, y via reference.
long long extgcd(long long a, long long b, long long &x, long
long &y) {
    if (b == 0) {
        x = 1;
        y = 0;
        return a;
    }

```

```

    }
    long long x1, y1;
    long long d = extgcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - y1 * (a / b);
    return d;
}

// 2. Solve Linear Diophantine Equation: ax + by = c
// Returns true if a solution exists, false otherwise.
// If true, (x0, y0) stores one valid base solution, and g
stores gcd(a, b).
//
// GENERAL SOLUTION GENERATOR:
// All solutions can be generated using an integer k:
// x = x0 + k * (b / g)
// y = y0 - k * (a / g)
bool solve_diophantine(long long a, long long b, long long c,
long long &x0, long long &y0, long long &g) {
    // Solve for abs(a) and abs(b) first to maintain correct
signs
    g = extgcd(abs(a), abs(b), x0, y0);

    if (c % g != 0) {
        return false; // No solution if c is not divisible by
gcd(a, b)
    }

    x0 *= c / g;
    y0 *= c / g;

    // Adjust signs back
    if (a < 0) x0 = -x0;
    if (b < 0) y0 = -y0;

    return true;
}

// 3. Modular Multiplicative Inverse
// Solves: ax = 1 (mod m).
// Valid ONLY IF a and m are coprime (gcd(a, m) == 1).
// Returns -1 if the inverse does not exist.
long long mod_inverse(long long a, long long m) {
    long long x, y;
    long long g = extgcd(a, m, x, y);

    if (g != 1) {
        return -1; // Inverse doesn't exist
    }

    // x can be negative, so we adjust it to be in the range [0,
m - 1]
    return (x % m + m) % m;
}

```

7 Geometry

8 Miscellaneous